

1. Deepest Point:

Top

factorial(1)

n = 1

Return address: factorial(2)

Dynamic link: factorial(2)

Static link: factorial(2)

factorial(2)

n = 2

Return address: factorial(3)

Dynamic link: factorial(3)

Static link: factorial(3)

factorial(3)

n = 3

Return address: factorial(4)

Dynamic link: factorial(4)

Static link: factorial(4)

factorial(4)

n = 4

Return address: original call

Dynamic link: global

Static link: global

Bottom

Unwinding:

Top

factorial(2) $\rightarrow 2 * 1 = 2$

factorial(3)

factorial(4)

Bottom

Top

factorial(3) $\rightarrow 3 * 2 = 6$

factorial(4)

Bottom

Top

factorial(4) -> 4 * 6 = 24

Bottom

Top

[empty stack]

Bottom

2. a.

```
func makeMessage() {  
    let message = 5;  
  
    func showMessage() {  
        return message;  
    }  
  
    call helper(showMessage);  
}
```

```
func helper(f) {  
    call f();  
}
```

call makeMessage();

b. showMessage is created inside makeMessage, so its static link points to makeMessage. However, since showMessage is called by helper, its dynamic link points to helper. This means that the two links do not always point to the same place. The only reason static link is needed is so showMessage can access variables from the scope it was defined, while the dynamic link only shows which function called it.

3.
 - a. If we let S = total stack size in bytes, R = size of one activation record in bytes, and d = recursion depth, then recursion is safe as long as $dR \leq S$. Since that is the case, $d \leq S / R$ is the maximum safe recursion depth and a stack overflow occurs when $dR > S$.
 - b. A tail-recursive function can still cause stack overflow in a simple interpreter because every recursive call still adds another activation record. Since the old frame is not reused, the stack keeps getting bigger. Some compilers can optimize tail recursion by reusing the same frame instead.